Our Database with Sequelize

the 9 model:generate commands, the structure they create, and the ER diagram

Dinesh Rawat

Instructor · Shorter Loop × Snapied

> Today we did not write a single API yet. We did something that has to come first: we told the database what our data looks like. Nine tables, the columns in each, and how they connect. We used one command per table, and each command wrote two files for us. Read these notes tonight, run every command yourself, and try to break it. Tomorrow we build the real APIs on top of what you make tonight.

1 What we actually did today

We ran nine commands. Each one looks like this:

```
npx sequelize-cli model:generate --name User --attributes name:string,email:string
```

And each one created two files at once:

- a model in `models/`, the JavaScript object your code will talk to
- a migration in `migrations/`, the instructions that build the real table

> **Model vs migration, the difference that confuses everyone at first**
>
> The `migration` is a set of instructions: "make a table called users with these columns." You run it once and the table exists. The `model` is how your running app reads and writes rows in that table every day. Migration builds the shelf; model puts things on and off the shelf. One command wrote both, already matching each other.

2 The nine commands, in order

Run them top to bottom. The order is not random, it follows what depends on what (more on that in section 5).

the commands we ran

```
npx sequelize-cli model:generate --name User \
  --attributes name:string,email:string,password:string,tier:enum:'{free,pro}',aiCredits:integer

npx sequelize-cli model:generate --name Template \
  --attributes name:string,config:text

npx sequelize-cli model:generate --name Document \
  --attributes title:string,type:enum:'{resume,cover_letter}',userId:integer,templateId:integer

npx sequelize-cli model:generate --name Section \
  --attributes heading:string,position:integer,documentId:integer

npx sequelize-cli model:generate --name Item \
  --attributes content:text,position:integer,sectionId:integer

npx sequelize-cli model:generate --name Version \
  --attributes snapshot:text,label:string,documentId:integer

npx sequelize-cli model:generate --name Application \
  --attributes company:string,role:string,\
  status:enum:'{saved,applied,interview,offer,rejected}',userId:integer,documentId:integer

npx sequelize-cli model:generate --name Share \
  --attributes slug:string,documentId:integer

npx sequelize-cli model:generate --name Export \
  --attributes format:enum:'{pdf,docx}',fileUrl:string,documentId:integer,userId:integer
```

When each command runs it prints two lines: New model was created and New migration was created. If you see those two lines nine times each, you did it right.

3 Reading one command, piece by piece

Take the User command apart so the rest make sense:

Piece	What it means
`npx sequelize-cli`	run the Sequelize command-line tool
`model:generate`	the job: make a model and its migration
`--name User`	the model is called User (singular, capital)
`--attributes`	the columns follow, as name:type pairs
`name:string`	a column "name" that holds text
`tier:enum:'{free,pro}'`	a column that may only be "free" or "pro", nothing else
`aiCredits:integer`	a column that holds a whole number

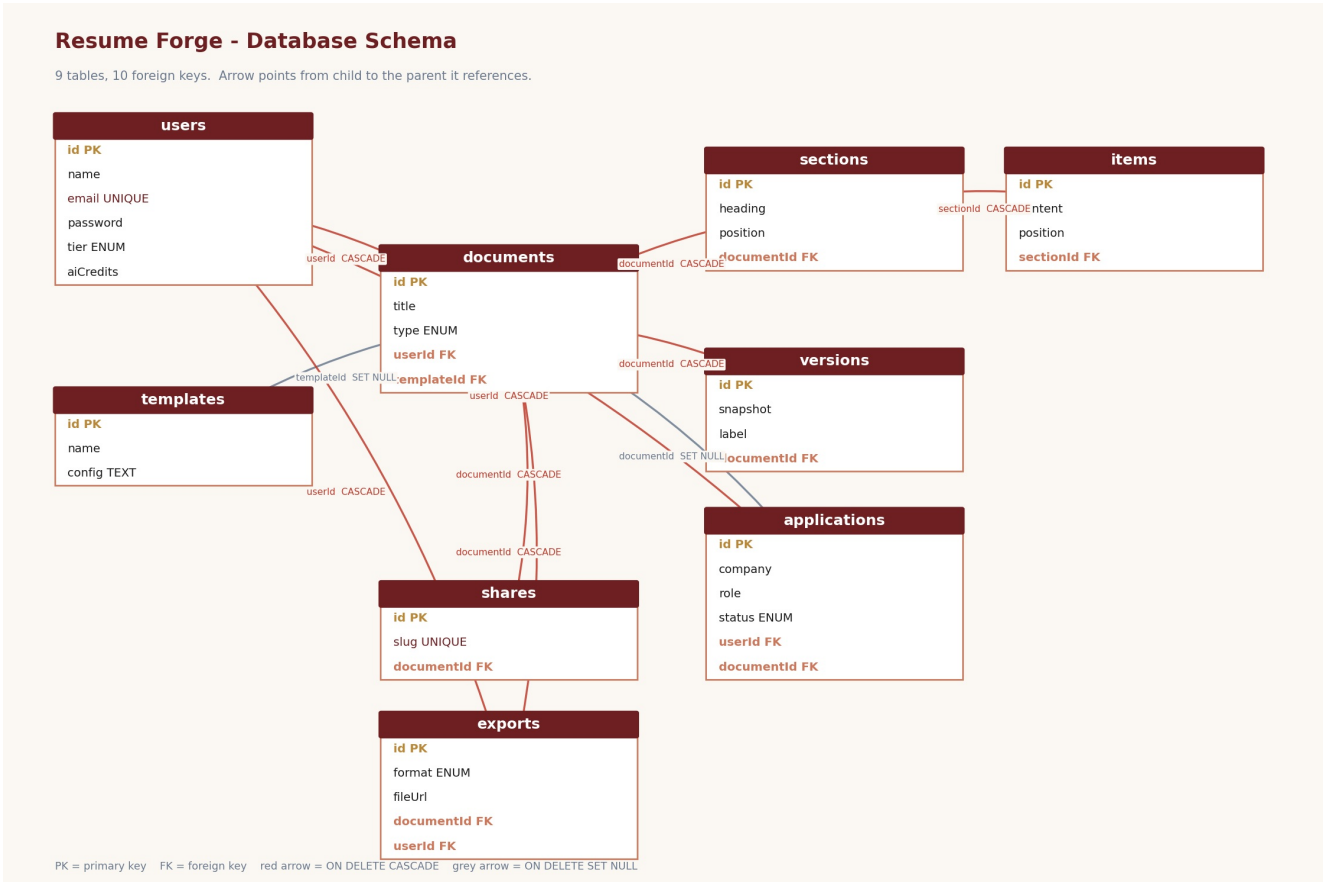
The column types we used. `string` is short text (a name, an email). `text` is long text (a whole snapshot, a config). `integer` is a whole number. `enum` is a fixed list of allowed words, the database itself rejects anything not on the list. That is why status can never accidentally become "aplied" with a typo, the enum will not allow it.

4 The nine tables, and what each holds

Table	What it stores	Points to
users	the account: name, email, password, tier, AI credits	nothing (top of the tree)
templates	a design a resume can use	nothing
documents	one resume or cover letter, the central table	users, templates
sections	a block in a document (Experience, Skills)	documents
items	one line inside a section (a single job)	sections
versions	a saved snapshot of a document	documents
applications	a tracked job application	users, documents
shares	a public link to a document	documents
exports	a generated PDF or DOCX record	documents, users

5 How the tables connect: the ER diagram

This is the whole database in one picture. An arrow points from a table to the one it depends on. For example, a section belongs to a document, so sections points to documents.



Reading the diagram

Gold rows are the primary key (id), the unique number for each row. Terracotta rows are foreign keys, a column that holds the id of a row in another table. That is the whole trick of a relational database: a section does not copy the document, it just stores the document's id. Follow the arrow and you find the real thing.

6 What CASCADE and SET NULL mean

Look at the arrow colours. Each foreign key has a rule for what happens when the parent is deleted.

CASCADE (red arrows): the child goes too. Delete a user and all their documents, sections, items, versions, applications, shares and exports are deleted with them. It makes sense, if the account is gone, its resumes should not linger. A document's sections and items cascade the same way.

SET NULL (grey arrows): the child survives, the link is cleared. Delete a template and any document using it is not deleted, its templateId is just set to empty. The resume lives on with no design attached. Same for an application pointing at a document you delete, the application stays in your tracker, it just loses the link.

These rules are decisions about your data. We chose each one on purpose. When you build tonight, read each foreign key and ask: if the parent disappears, should this die with it, or survive?

7 The two things the command does NOT do for you

This is the part to watch tonight. `model:generate` is a good start but it leaves two jobs for you.

1. Foreign keys start as plain numbers. When you write `userId:integer`, the tool makes an ordinary number column. It does not know it should point at the users table. You open the migration and add the link yourself:

```
userId: {
  type: Sequelize.INTEGER,
  references: { model: 'Users', key: 'id' },
  onUpdate: 'CASCADE',
  onDelete: 'CASCADE'
}
```

2. The model's associate() is empty. The tool leaves a blank `associate()` in every model. You fill in the relationships so joins work:

```
Document.belongsTo(models.User,  { foreignKey: 'userId' });
Document.hasMany(models.Section, { foreignKey: 'documentId', onDelete: 'CASCADE' });
```

8 The order trap (this will bite you)

Migrations run in filename order, oldest first, and a table cannot point at a table that does not exist yet. The tool stamps each file with the time down to the second, so if two commands run in the same second the order can come out wrong. If `documents` tries to run before `users`, the foreign key fails because there is no users table yet. On MySQL this is a hard error. Check your filenames run in this order, and rename them if they do not:

users → templates → documents → sections → items → versions → applications → shares → exports

9 Building and undoing the tables

```
npx sequelize-cli db:migrate            # build every table
npx sequelize-cli db:migrate:undo       # undo the last table
npx sequelize-cli db:migrate:undo:all   # undo everything
npx sequelize-cli db:migrate:status     # see what has run
```

Migrations are safe to undo. Unlike deleting a database by hand, every table you build is one command away from being removed and rebuilt. This is why teams use migrations, the whole structure lives in files you can replay on any machine and get the identical result.

10 Your homework: read, try, break

Run all nine commands in a fresh project (npm install sequelize sequelize-cli mysql2, then npx sequelize-cli init, then the nine commands). Watch the two files appear each time.

Open the files and read them. Find the empty associate(). Find the plain userId column. See for yourself what the tool did and did not do.

Add the foreign keys and associations from sections 7. Then run db:migrate and open your database, are all nine tables there?

Now break it on purpose. Rename a migration so documents runs before users and watch it fail, read the error. Undo everything with db:migrate:undo:all and build it again. Put a wrong value in an enum column and see the database refuse it.

Come tomorrow with it working, or with the exact error you got stuck on. Tomorrow we write the real APIs on top of these tables. The students who broke it and fixed it tonight will fly through that. Watching is not building, type it, run it, break it, fix it.

Everything here ran live in class. If a part does not click, message me the file name and the line, and read it again tonight while it is fresh.